CSE 276A HW1: Waypoint Navigation

Fong Yu Lin (200057839), Vanshik Nikunj Waghela (200478355)

16 October 2026

1 Video

`https://youtu.be/189XVaLgCpw?si=ek8AVvpiOvj4hXzN`

2 What is the challenge

The rover has to drive autonomously through these waypoints (x, y, θ), in meters and radians:

$$(0,0,0) \to (1,0,0) \to (1,1,1.57) \to (2,1,0) \to (2,2,-1.57) \to (1,1,-0.78) \to (0,0,0)$$

x is forward from the start, y is to the left, and θ is the heading, we set that the counter-clockwise is positive. For this robot, we can only send it two wheel commands $(L, R) \in [-1, 1]$ on `/motor_commands`, and the only sensor I get back is an IMU on `/imu_data` which does not work accurately. There are no wheel encoders and no camera feedback.

We found it's hard to let the robot drive autonomously when the robot itself has no idea where it is. A wheel command is not a speed, there is a deadband where nothing moves (probably due to high friction), the two wheels are not identical, and we have to get from "send numbers to the motors" to "end up at $(2, 2)$ facing $-90°$".

3 Algorithm

We decided to use the strategy **turn** $\to$ **drive** $\to$ **turn**. If the robot is at (x, y) and the next waypoint is (x_g, y_g, θ_g):

$$d = \sqrt{(x_g - x)^2 + (y_g - y)^2}, \qquad \phi = \operatorname{atan2}(y_g - y,\ x_g - x), \qquad e = \operatorname{wrap}(\phi - \hat{\theta})$$

where d is the distance to drive, ϕ is the direction to face before driving, $\hat{\theta}$ is my current heading estimate, and e is the heading error. $\operatorname{wrap}(a) = \operatorname{atan2}(\sin a, \cos a)$ keeps angles in $(-\pi, \pi]$ so the robot always turns the short way (e.g. $350°$ becomes $-10°$). Then:

1. Turn in place until the heading is ϕ.
2. Drive straight for d meters while holding heading ϕ.
3. Turn in place until the heading is θ_g.

After each run, we just assume the robot reached (x_g, y_g) and use it as the start of the next run. We picked this over driving curves because each motion only needs one kind of calibration (straight speed or turning), and every waypoint in this task is reachable with straight lines anyway.

4 What is missing, and how we determined it (calibration)

The algorithm needs two things the robot doesn't give us: **how far it went** and **which way it is facing**. We wrote `calibrate.py`, which runs fixed (L, R) for a fixed time and logs the IMU, then measured the result with a ruler and our eyes.

Distance: from time. With no encoders, distance is open loop: $t = d/v$. We start by testing certain speed commands for certain amount of times, how far does the robot reach and turn.

- 0.2 for 2 s: 42, 42.5, 43 cm $\Rightarrow$ 0.213 m/s.
- 0.3 for 2 s: 76, 75, 75 cm $\Rightarrow v = 0.377$ m/s. Check: 2.65 s drove exactly 1.00 m.
- A line through these gives $v \approx 1.64\,u - 0.116$ (u = wheel command), so the deadband is about $u \approx 0.07$. Indeed 0.07 didn't move and 0.08 barely did.
- Without feedback it drifts left, so the right wheel is a bit faster.

We use $u = 0.3$ for driving: fast enough to be well out of the deadband, slow enough to be repeatable.

Turning: the command has to be big. In place $(-u, +u)$: 0.3 barely moves, 0.35 is inconsistent, 0.4–0.45 often won't start from rest, and 0.5 turns reliably ($\sim 190^\circ$ in 1 s). So we turn at 0.5.

Heading: which sensor to trust. The IMU gives a fused yaw, but it was clearly wrong while turning: it read 502° for a $\sim 190^\circ$ turn, gave 348° and 19° on two runs of the same test, and drifted back $\sim 30^\circ$ after the robot stopped (probably the motors messing with the magnetometer). The raw gyro z rate was fine, so we integrate it ourselves:

$$\hat{\theta}_{k+1} = \mathrm{wrap}\big(\hat{\theta}_k + s\,(\omega_k - b)\,\Delta t\big)$$

where ω_k is the gyro z reading (converted to rad/s), b is the gyro bias averaged over 2 s standing still at startup, Δt is the time between IMU samples, and s is a scale factor. Turning 4 s at $(0, 0.5)$ gave 369.1°, 377.0°, 379.3° from the gyro, matching what we measured on the floor, so $s = 1.0$.

A timing bug we found. Our first turn test was off by up to +24°, and on one run the gyro said +3.8° while the robot was really $\sim 50^\circ$ off. The cause was `motor_control.py` reading the serial port with `read(1024)`, which delivered IMU messages in bursts of 4–5 every ~ 0.09 s. Using each message's arrival gap as Δt was then wrong, and it added ~ 0.1 s of latency. We changed it to `read(in_waiting or 1)` and use a running average $\Delta t \leftarrow 0.98\,\Delta t + 0.02$ gap (about 48.5 Hz).

5 Resulting solution

Turning. Turn at ± 0.5 until $|e| < 15^\circ$ (the lead angle makes up for the robot coasting after it stops), wait 0.3 s, then nudge with 0.12 s pulses until $|e| < 5^\circ$. It gives up after 8 steps or 5 s so it can't spin forever.

Driving straight (P control on heading). Drive for $t = d/v$ seconds while correcting heading:

$$c = \mathrm{clip}(K_p\,e,\ -0.1,\ 0.1), \qquad L = u - c, \qquad R = u + c$$

where $u = 0.3$ is the base command, e is the heading error, $K_p = 0.3$, and c is the correction. If the robot drifts right ($e > 0$), the right wheel speeds up and the left slows down, which steers it back. The clip keeps one wheel from dropping into the deadband.

Parameter	Value	Parameter	Value
Drive command u	0.3	Turn command	0.5
Drive speed v	0.377 m/s	Turn lead / tolerance	15° / 5°
K_p / max correction	0.3 / 0.1	Pulse / settle time	0.12 s / 0.3 s

Code (github.com/YangLin14/rubikpi_ros2, package `robot_control`):

- `waypoint_navigator.py`: the whole algorithm above. Subscribes to `/imu_data`, publishes `/motor_commands`, and can read waypoints from a text file.

- `calibrate.py`: runs (L, R) for a set time and prints gyro and yaw turn angles.
- `ssh_keyboard_control.py`: hold-to-drive teleop that works over SSH.
- `motor_control.py`: the serial read fix.

6 Test results

We tested in steps before running the full path:

Test	Result	What we changed
First run, 1 m	1.00 m in 2.65 s, very straight, $-2.8°$	Nothing, it worked
Turns v1	Errors $+23.6°$, $+11.8°$, $-3.0°$, $-3.8°$	Turn cmd 0.5, 15° lead, serial/Δt fix
Turns v2	$-1.5°$, $-0.4°$, $-1.0°$, $-0.1°$, but pulses overshot back and forth	Tolerance $4° \to 5°$, pulse $0.15 \to 0.12$ s
Turns v3	$+2.3°$, $+1.5°$, $-2.5°$, $+1.8°$, no oscillation	Final settings

Full HW1 run (gyro bias $-0.63°$/s). Heading at each waypoint from the gyro:

Waypoint	Target θ	Final heading	Error
$(1, 0)$	$0°$	$+2.9°$	$-2.9°$
$(1, 1)$	$+90°$	$+93.1°$	$-3.1°$
$(2, 1)$	$0°$	$+2.9°$	$-2.9°$
$(2, 2)$	$-90°$	$-94.0°$	$+4.1°$
$(1, 1)$	$-45°$	$-45.4°$	$+0.7°$
$(0, 0)$	$0°$	$-4.0°$	$+4.0°$

All six headings ended within $\pm 4.1°$, and the robot came back close to the start. Total driving time was 18.1 s (four 1 m legs at 2.65 s and two $\sqrt{2}$ m legs at 3.75 s).

Comments on performance.

- **Turns are good.** Gyro integration plus pulses gets within the 5° tolerance every time.
- **Straight legs drift about $+3°$ left.** This is the steady-state error of P control: the right wheel is $\sim 10\%$ faster, so the robot needs a constant correction $c \approx 0.016$, and P control can only make that if an error $e = c/K_p \approx 3°$ stays. Adding an integral term (PI) would remove it.
- **Position is open loop.** Only heading has feedback. Distance comes from time $\times$ speed, so errors from wheel slip, battery level, and the floor add up over the path, and the robot can't tell. Gyro drift also adds up because the heading is never reset by an absolute reference.
- **Improvements:** PI control while driving, recalibrating speed when the battery drops, and an absolute sensor (camera landmarks) to correct position.

7 Credits

We used Claude to help us learn and complete this assignment.